CHAPTER 3

Transition-Based Parsing

In data-driven dependency parsing, the goal is to learn a good predictor of dependency trees, that is, a model that can be used to map an input sentence $S = w_0 w_1 \ldots w_n$ to its correct dependency tree G. As explained in the previous chapter, such a model has the general form $M = (\Gamma, \boldsymbol{\lambda}, h)$, where Γ is a set of constraints that define the space of permissible structures for a given sentence, $\boldsymbol{\lambda}$ is a set of parameters, the values of which have to be learned from data, and h is a fixed parsing algorithm. In this chapter, we are going to look at systems that parameterize a model over the transitions of an abstract machine for deriving dependency trees, where we learn to predict the next transition given the input and the parse history, and where we predict new trees using a greedy, deterministic parsing algorithm – this is what we call transition-based parsing. In chapter 4, we will instead consider systems that parameterize a model over sub-structures of dependency trees, where we learn to score entire dependency trees given the input, and where we predict new trees using exact inference – graph-based parsing. Since most transition-based and graph-based systems do not make use of a formal grammar at all, Γ will typically only restrict the possible dependency trees for a sentence to those that satisfy certain formal constraints, for example, the set of all projective trees (over a given label set). In chapter 5, by contrast, we will deal with grammar-based systems, where Γ constitutes a formal grammar pairing each input sentence with a more restricted (possibly empty) set of dependency trees.

3.1 TRANSITION SYSTEMS

A *transition system* is an abstract machine, consisting of a set of *configurations* (or *states*) and *transitions* between configurations. One of the simplest examples is a finite state automaton, which consists of a finite set of atomic states and transitions defined on states and input symbols, and which accepts an input string if there is a sequence of valid transitions from a designated *initial* state to one of several *terminal* states. By contrast, the transition systems used for dependency parsing have complex configurations with internal structure, instead of atomic states, and transitions that correspond to steps in the derivation of a dependency tree. The idea is that a sequence of valid transitions, starting in the initial configuration for a given sentence and ending in one of several terminal configurations, defines a valid dependency tree for the input sentence. In this way, the transition system determines the constraint set Γ in the parsing model, since it implicitly defines the set of permissible dependency trees for a given sentence, but it also determines the parameter set $\boldsymbol{\lambda}$ that have to be learned from data, as we shall see later on. For most of this chapter, we will concentrate on a simple stack-based transition system, which implements a form of shift-reduce parsing and exemplifies the most widely

used approach in transition-based dependency parsing. In section 3.4, we will briefly discuss some of the alternative systems that have been proposed.

We start by defining configurations as triples consisting of a stack, an input buffer, and a set of dependency arcs.

Definition 3.1. Given a set R of dependency types, a *configuration* for a sentence $S = w_0 w_1 \dots w_n$ is a triple $c = (\sigma, \beta, A)$, where

1. σ is a stack of words $w_i \in V_S$,
2. β is a buffer of words $w_i \in V_S$,
3. A is a set of dependency arcs $(w_i, r, w_j) \in V_S \times R \times V_S$.

The idea is that a configuration represents a partial analysis of the input sentence, where the words on the stack σ are partially processed words, the words in the buffer β are the remaining input words, and the arc set A represents a partially built dependency tree. For example, if the input sentence is

Economic news had little effect on financial markets.

then the following is a valid configuration, where the stack contains the words ROOT and *news* (with the latter on top), the buffer contains all the remaining words except *Economic*, and the arc set contains a single arc connecting the head *news* to the dependent *Economic* with the label ATT:

$$([\text{ROOT}, \text{news}]_\sigma, [\text{had}, \text{little}, \text{effect}, \text{on}, \text{financial}, \text{markets}, .]_\beta, \{(\text{news}, \text{ATT}, \text{Economic})\}_A)$$

Note that we represent both the stack and the buffer as simple lists, with elements enclosed in square brackets (and subscripts σ and β when needed), although the stack has its head (or top) to the right for reasons of perspicuity. When convenient, we use the notation $\sigma|w_i$ to represent the stack which results from pushing w_i onto the stack σ, and we use $w_i|\beta$ to represent a buffer with head w_i and tail β.[1]

Definition 3.2. For any sentence $S = w_0 w_1 \dots w_n$,

1. the *initial* configuration $c_0(S)$ is $([w_0]_\sigma, [w_1, \dots, w_n]_\beta, \emptyset)$,
2. a *terminal* configuration is a configuration of the form $(\sigma, [\,]_\beta, A)$ for any σ and A.

Thus, we initialize the system to a configuration with w_0 = ROOT on the stack, all the remaining words in the buffer, and an empty arc set; and we terminate in any configuration that has an empty buffer (regardless of the state of the stack and the arc set).

[1]The operator | is taken to be left-associative for the stack and right-associative for the buffer.

Transition		Precondition
LEFT-ARC$_r$	$(\sigma \mid w_i, w_j \mid \beta, A) \Rightarrow (\sigma, w_j \mid \beta, A \cup \{(w_j, r, w_i)\})$	$i \neq 0$
RIGHT-ARC$_r$	$(\sigma \mid w_i, w_j \mid \beta, A) \Rightarrow (\sigma, w_i \mid \beta, A \cup \{(w_i, r, w_j)\})$	
SHIFT	$(\sigma, w_i \mid \beta, A) \Rightarrow (\sigma \mid w_i, \beta, A)$	

Figure 3.1: Transitions for shift-reduce dependency parsing.

Having defined the set of configurations, including a unique initial configuration and a set of terminal configurations for any sentence, we now define transitions between configurations. Formally speaking, a transition is a partial function from configurations to configurations, i.e., a transition maps a given configuration to a new configuration but may be undefined for certain configurations. Conceptually, a transition corresponds to a basic parsing action that adds an arc to the dependency tree or modifies the stack or the buffer. The transitions needed for shift-reduce dependency parsing are defined in figure 3.1 and contain three types of transitions:

1. Transitions LEFT-ARC$_r$ (for any dependency label r) add a dependency arc (w_j, r, w_i) to the arc set A, where w_i is the word on top of the stack and w_j is the first word in the buffer. In addition, they pop the stack. They have as precondition that both the stack and the buffer are non-empty and that $w_i \neq$ ROOT.[2]

2. Transitions RIGHT-ARC$_r$ (for any dependency label r) add a dependency arc (w_i, r, w_j) to the arc set A, where w_i is the word on top of the stack and w_j is the first word in the buffer. In addition, they pop the stack and replace w_j by w_i at the head of buffer.[3] They have as their only precondition that both the stack and the buffer are non-empty.

3. The transition SHIFT removes the first word w_i in the buffer and pushes it on top of the stack. It has as its only precondition that the buffer is non-empty.

We use the symbol $\mathcal{T}$ to refer to the set of permissible transitions in a given transition system. As noted above, transitions correspond to elementary parsing actions. In order to define complete parses, we introduce the notion of a *transition sequence*.

Definition 3.3. A *transition sequence* for a sentence $S = w_0 w_1 \ldots w_n$ is a sequence of configurations $C_{0,m} = (c_0, c_1, \ldots, c_m)$ such that

[2]The latter precondition guarantees that the dependency graph defined by the arc set always satisfies the root property.
[3]This may seem counterintuitive, given that the buffer is meant to contain words that have not yet been processed, but it is necessary in order to allow w_j to attach to a head on its left.

Transition	Configuration		
	([ROOT],	[Economic, . . . , .],	∅)
SH ⇒	([ROOT, Economic],	[news, . . . , .],	∅)
LA_{ATT} ⇒	([ROOT],	[news, . . . , .],	$A_1 = \{(\text{news, ATT, Economic})\}$)
SH ⇒	([ROOT, news],	[had, . . . , .],	A_1)
LA_{SBJ} ⇒	([ROOT],	[had, . . . , .],	$A_2 = A_1 \cup \{(\text{had, SBJ, news})\}$)
SH ⇒	([ROOT, had],	[little, . . . , .],	A_2)
SH ⇒	([ROOT, had, little],	[effect, . . . , .],	A_2)
LA_{ATT} ⇒	([ROOT, had],	[effect, . . . , .],	$A_3 = A_2 \cup \{(\text{effect, ATT, little})\}$)
SH ⇒	([ROOT, had, effect],	[on, . . . , .],	A_3)
SH ⇒	([ROOT, . . . on],	[financial, markets, .],	A_3)
SH ⇒	([ROOT, . . . , financial],	[markets, .],	A_3)
LA_{ATT} ⇒	([ROOT, . . . on],	[markets, .],	$A_4 = A_3 \cup \{(\text{markets, ATT, financial})\}$)
RA_{PC} ⇒	([ROOT, had, effect],	[on, .],	$A_5 = A_4 \cup \{(\text{on, PC, markets})\}$)
RA_{ATT} ⇒	([ROOT, had],	[effect, .],	$A_6 = A_5 \cup \{(\text{effect, ATT, on})\}$)
RA_{OBJ} ⇒	([ROOT],	[had, .],	$A_7 = A_6 \cup \{(\text{had, OBJ, effect})\}$)
SH ⇒	([ROOT, had],	[.],	A_7)
RA_{PU} ⇒	([ROOT],	[had],	$A_8 = A_7 \cup \{(\text{had, PU, .})\}$)
RA_{PRED} ⇒	([ ],	[ROOT],	$A_9 = A_8 \cup \{(\text{ROOT, PRED, had})\}$)
SH ⇒	([ROOT],	[ ],	A_9)

Figure 3.2: Transition sequence for the English sentence in figure 1.1 (LA_r = LEFT-ARC$_r$, RA_r = RIGHT-ARC$_r$, SH = SHIFT).

1. c_0 is the initial configuration $c_0(S)$ for S,
2. c_m is a terminal configuration,
3. for every i such that $1 \leq i \leq m$, there is a transition $t \in \mathcal{T}$ such that $c_i = t(c_{i-1})$.

A transition sequence starts in the initial configuration for a given sentence and reaches a terminal configuration by applying valid transitions from one configuration to the next. The dependency tree derived through this transition sequence is the dependency tree defined by the terminal configuration, i.e., the tree $G_{c_m} = (V_S, A_{c_m})$, where A_{c_m} is the arc set in the terminal configuration c_m. By way of example, figure 3.2 shows a transition sequence that derives the dependency tree shown in figure 1.1 on page 2.

The transition system defined for dependency parsing in this section leads to derivations that correspond to basic shift-reduce parsing for context-free grammars. The LEFT-ARC$_r$ and RIGHT-ARC$_r$ transitions correspond to reduce actions, replacing a head-dependent structure with its head, while the SHIFT transition is exactly the same as the shift action. One peculiarity of the transitions, as defined here, is that the "reduce transitions" apply to one node on the stack and one node in the buffer, rather than two nodes on the stack. This simplifies the definition of terminal configurations and has become standard in the dependency parsing literature.

Every transition sequence in this system defines a dependency graph with the *spanning*, *root*, and *single-head* properties, but not necessarily with the *connectedness* property. This means that not every transition sequence defines a dependency tree, as defined in chapter 2. To take a trivial example, a transition sequence for a sentence S consisting only of SHIFT transitions defines the graph $G = (V_S, \emptyset)$, which is not connected but which satisfies all the other properties. However, since any transition sequence defines an *acyclic* dependency graph G, it is trivial to convert G into a dependency tree G' by adding arcs of the form (ROOT, r, w_i) (with some dependency label r) for every w_i that is a root in G. As noted in section 2.1.1, a dependency graph G that satisfies the spanning, root, single-head, and acyclic properties is equivalent to a set of dependency trees and is often called a *dependency forest*.

Another important property of the system is that every transition sequence defines a *projective* dependency forest,[4] which is advantageous from the point of view of efficiency but overly restrictive from the point of view of representational adequacy. In sections 3.4 and 3.5, we will see how this limitation can be overcome, either by modifying the transition system or by complementing it with pre- and post-processing.

Given that every transition sequence defines a projective dependency forest, which can be turned into a dependency tree, we say that the system is *sound* with respect to the set of projective dependency trees. A natural question is whether the system is also *complete* with respect to this class of dependency trees, that is, whether every projective dependency tree is defined by some transition sequence. The answer to this question is affirmative, although we will not prove it here.[5] In terms of our parsing model $M = (\Gamma, \boldsymbol{\lambda}, h)$, we can therefore say that the transition system described in this section corresponds to a set of constraints Γ characterizing the set $\mathcal{G}_S^p$ of projective dependency trees for a given sentence S (relative to a set of arc labels R).

3.2 PARSING ALGORITHM

The transition system defined in section 3.1 is nondeterministic in the sense that there is usually more than one transition that is valid for any non-terminal configuration.[6] Thus, in order to perform deterministic parsing, we need a mechanism to determine for any non-terminal configuration c, what is the correct transition out of c. Let us assume for the time being that we are given an *oracle*, that is, a function o from configurations to transitions such that $o(c) = t$ if and only if t is the correct transition out of c. Given such an oracle, deterministic parsing can be achieved by the very simple algorithm in figure 3.3.

We start in the initial configuration $c_0(S)$ and, as long as we have not reached a terminal configuration, we use the oracle to find the optimal transition $t = o(c)$ and apply it to our current configuration to reach the next configuration $t(c)$. Once we reach a terminal configuration, we simply return the dependency tree defined by our current arc set. Note that, while finding the

[4] A dependency forest is projective if and only if all component trees are projective.
[5] The interested reader is referred to Nivre (2008) for proofs of soundness and completeness for this and several other transition systems for dependency parsing.
[6] The notable exception is a configuration with an empty stack, where only SHIFT is possible.

h(S, Γ, o)
1 $c \leftarrow c_0(S)$
2 **while** c is not terminal
3 $\quad t \leftarrow o(c)$
4 $\quad c \leftarrow t(c)$
5 **return** G_c

Figure 3.3: Deterministic, transition-based parsing with an oracle.

optimal transition $t = o(c)$ is a hard problem, which we have to tackle using machine learning, computing the next configuration $t(c)$ is a purely mechanical operation.

It is easy to show that, as long as there is at least one valid transition for every non-terminal configuration, such a parser will construct exactly one transition sequence $C_{0,m}$ for a sentence S and return the dependency tree defined by the terminal configuration c_m, i.e., $G_{c_m} = (V_S, A_{c_m})$. To see that there is always at least one valid transition out of a non-terminal configuration, we only have to note that such a configuration must have a non-empty buffer (otherwise it would be terminal), which means that at least SHIFT is a valid transition.

The time complexity of the deterministic, transition-based parsing algorithm is $O(n)$, where n is the number of words in the input sentence S, provided that the oracle and transition functions can be computed in constant time. This holds since the worst-case running time is bounded by the maximum number of transitions in a transition sequence $C_{0,m}$ for a sentence $S = w_0 w_1 \ldots w_n$. Since a SHIFT transition decreases the length of the buffer by 1, no other transition increases the length of the buffer, and any configuration with an empty buffer is terminal, the number of SHIFT transitions in $C_{0,m}$ is bounded by n. Moreover, since both LEFT-ARC$_r$ and RIGHT-ARC$_r$ decrease the height of the stack by 1, only SHIFT increases the height of the stack by 1, and the initial height of the stack is 1, the combined number of instances of LEFT-ARC$_r$ and RIGHT-ARC$_r$ in $C_{0,m}$ is also bounded by n. Hence, the worst-case time complexity is $O(n)$.

So far, we have seen how transition-based parsing can be performed in linear time if restricted to projective dependency trees, and provided that we have a constant-time oracle that predicts the correct transition out of any non-terminal configuration. Of course, oracles are hard to come by in real life, so in order to build practical parsing systems, we need to find some other mechanism that we can use to approximate the oracle well enough to make accurate parsing feasible. There are many conceivable ways of approximating oracles, including the use of formal grammars and disambiguation heuristics. However, the most successful strategy to date has been to take a data-driven approach, approximating oracles by *classifiers* trained on treebank data. This leads to the notion of classifier-based parsing, which is an essential component of transition-based dependency parsing.

h$(S, \Gamma, \boldsymbol{\lambda})$
1 $c \leftarrow c_0(S)$
2 **while** c is not terminal
3 $t \leftarrow \lambda_c$
4 $c \leftarrow t(c)$
5 **return** G_c

Figure 3.4: Deterministic, transition-based parsing with a classifier.

3.3 CLASSIFIER-BASED PARSING

Let us step back for a moment to our general characterization of a data-driven parsing model as $M = (\Gamma, \boldsymbol{\lambda}, h)$, where Γ is a set of constraints on dependency graphs, $\boldsymbol{\lambda}$ is a set of model parameters and h is a fixed parsing algorithm. In the previous two sections, we have shown how we can define the parsing algorithm h as deterministic best-first search in a transition system (although other search strategies are possible, as we shall see later on). The transition system determines the set of constraints Γ, but it also defines the model parameters $\boldsymbol{\lambda}$ that need to be learned from data, since we need to be able to predict the oracle transition $o(c)$ for every possible configuration c (for any input sentence S). We use the notation $\lambda_c \in \boldsymbol{\lambda}$ to denote the transition predicted for c according to model parameters $\boldsymbol{\lambda}$, and we can think of $\boldsymbol{\lambda}$ as a huge table containing the predicted transition λ_c for every possible configuration c. In practice, $\boldsymbol{\lambda}$ is normally a compact representation of a function for computing λ_c given c, but the details of this representation need not concern us now. Given a learned model, we can perform deterministic, transition-based parsing using the algorithm in figure 3.4, where we have simply replaced the oracle function o by the learned parameters $\boldsymbol{\lambda}$ (and the function value $o(c)$ by the specific parameter value λ_c).

However, in order to make the learning problem tractable by standard machine learning techniques, we need to introduce an abstraction over the infinite set of possible configurations. This is what is achieved by the feature function $\mathbf{f}(x) : \mathcal{X} \rightarrow \mathcal{Y}$ (cf. section 2.2). In our case, the domain $\mathcal{X}$ is the set $\mathcal{C}$ of possible configurations (for any sentence S) and the range $\mathcal{Y}$ is a product of m feature value sets, which means that the feature function $\mathbf{f}(c) : \mathcal{C} \rightarrow \mathcal{Y}$ maps every configuration to an m-dimensional feature vector. Given this representation, we then want to learn a classifier $g : \mathcal{Y} \rightarrow \mathcal{T}$, where $\mathcal{T}$ is the set of possible transitions, such that $g(\mathbf{f}(c)) = o(c)$ for any configuration c. In other words, given a training set of gold standard dependency trees from a treebank, we want to learn a classifier that predicts the oracle transition $o(c)$ for any configuration c, given as input the feature representation $\mathbf{f}(c)$. This gives rise to three basic questions:

- How do we represent configurations by feature vectors?
- How do we derive training data from treebanks?
- How do we train classifiers?

We will deal with each of these questions in turn, starting with feature representations in section 3.3.1, continuing with the derivation of training data in section 3.3.2, and finishing off with the training of classifiers in section 3.3.3.

3.3.1 FEATURE REPRESENTATIONS

A feature representation $\mathbf{f}(c)$ of a configuration c is an m-dimensional vector of simple features $\mathbf{f}_i(c)$ (for $1 \leq i \leq m$). In the general case, these simple features can be defined by arbitrary attributes of a configuration, which may be either categorical or numerical. For example, "the part of speech of the word on top of the stack" is a categorical feature, with values taken from a particular part-of-speech tagset (e.g., NN for noun, VB for verb, etc.). By contrast, "the number of dependents previously attached to the word on top of the stack" is a numerical feature, with values taken from the set $\{0, 1, 2, \ldots\}$. The choice of feature representations is partly dependent on the choice of learning algorithm, since some algorithms impose special restrictions on the form that feature values may take, for example, that all features must be numerical. However, in the interest of generality, we will ignore this complication for the time being and assume that features can be of either type. This is unproblematic since it is always possible to convert categorical features to numerical features, and it will greatly simplify the discussion of feature representations for transition-based parsing.

The most important features in transition-based parsing are defined by attributes of words, or tree nodes, identified by their position in the configuration. It is often convenient to think of these features as defined by two simpler functions, an *address function* identifying a particular word in a configuration (e.g., the word on top of the stack) and an *attribute function* selecting a specific attribute of this word (e.g., its part of speech). We call these features *configurational word features* and define them as follows.

Definition 3.4. Given an input sentence $S = w_0 w_1 \ldots w_n$ with node set V_S, a function $(v \circ a)(c) : \mathcal{C} \rightarrow Y$ composed of

1. an address function $a(c) : \mathcal{C} \rightarrow V_S$,
2. an attribute function $v(w) : V_S \rightarrow Y$.

is a *configurational word feature*.

An address function can in turn be composed of simpler functions, which operate on different components of the input configuration c. For example:

- Functions that extract the kth word (from the top) of the stack or the kth word (from the head) of the buffer.
- Functions that map a word w to its parent, leftmost child, rightmost child, leftmost sibling, or rightmost sibling in the dependency graph defined by c.

Table 3.1: Feature model for transition-based parsing.

f_i	Address	Attribute
1	STK[0]	FORM
2	BUF[0]	FORM
3	BUF[1]	FORM
4	LDEP(STK[0])	DEPREL
5	RDEP(STK[0])	DEPREL
6	LDEP(BUF[0])	DEPREL
7	RDEP(BUF[0])	DEPREL

By defining such functions, we can construct arbitrarily complex address functions that extract, e.g., "the rightmost sibling of the leftmost child of the parent of the word on top of the stack" although the address functions used in practice typically combine at most three such functions. It is worth noting that most address functions are partial, which means that they may fail to return a word. For example, a function that is supposed to return the leftmost child of the word on top of the stack is undefined if the stack is empty or if the word on top of the stack does not have any children. In this case, any feature defined with this address function will also be undefined (or have a special null value).

The typical attribute functions refer to some linguistic property of words, which may be given as input to the parser or computed as part of the parsing process. We can exemplify this with the word *markets* from the sentence in figure 1.1:

- Identity of w_i = *markets*
- Identity of lemma of w_i = *market*
- Identity of part-of-speech tag for w_i = NNS
- Identity of dependency label for w_i = PC

The first three attributes are *static* in the sense that they are constant, if available at all, in every configuration for a given sentence. That is, if the input sentence has been lemmatized and tagged for parts of speech in preprocessing, then the values of these features are available for all words of the sentence, and their values do not change during parsing. By contrast, the dependency label attribute is *dynamic* in the sense that it is available only after the relevant dependency arc has been added to the arc set. Thus, in the transition sequence in figure 3.2, the dependency label for the word *markets* is undefined in the first twelve configurations, but has the value PC in all the remaining configurations. Hence, such attributes can be used to define features of the transition history and the partially built dependency tree, which turns out to be one of the major advantages of the transition-based approach.

$\mathbf{f}(c_0)$	=	(ROOT	Economic	news	NULL	NULL	NULL	NULL)
$\mathbf{f}(c_1)$	=	(Economic	news	had	NULL	NULL	NULL	NULL)
$\mathbf{f}(c_2)$	=	(ROOT	news	had	NULL	NULL	ATT	NULL)
$\mathbf{f}(c_3)$	=	(news	had	little	ATT	NULL	NULL	NULL)
$\mathbf{f}(c_4)$	=	(ROOT	had	little	NULL	NULL	SBJ	NULL)
$\mathbf{f}(c_5)$	=	(had	little	effect	SBJ	NULL	NULL	NULL)
$\mathbf{f}(c_6)$	=	(little	effect	on	NULL	NULL	NULL	NULL)
$\mathbf{f}(c_7)$	=	(had	effect	on	SBJ	NULL	ATT	NULL)
$\mathbf{f}(c_8)$	=	(effect	on	financial	ATT	NULL	NULL	NULL)
$\mathbf{f}(c_9)$	=	(on	financial	markets	NULL	NULL	NULL	NULL)
$\mathbf{f}(c_{10})$	=	(financial	markets	.	NULL	NULL	NULL	NULL)
$\mathbf{f}(c_{11})$	=	(on	markets	.	NULL	NULL	ATT	NULL)
$\mathbf{f}(c_{12})$	=	(effect	on	.	ATT	NULL	NULL	ATT)
$\mathbf{f}(c_{13})$	=	(had	effect	.	SBJ	NULL	ATT	ATT)
$\mathbf{f}(c_{14})$	=	(ROOT	had	.	NULL	NULL	SBJ	OBJ)
$\mathbf{f}(c_{15})$	=	(had	.	NULL	SBJ	OBJ	NULL	NULL)
$\mathbf{f}(c_{16})$	=	(ROOT	had	NULL	NULL	NULL	SBJ	PU)
$\mathbf{f}(c_{17})$	=	(NULL	ROOT	NULL	NULL	NULL	NULL	PRED)
$\mathbf{f}(c_{18})$	=	(ROOT	NULL	NULL	NULL	PRED	NULL	NULL)

Figure 3.5: Feature vectors for the configurations in figure 3.2.

Let us now try to put all the pieces together and examine a complete feature representation using only configurational word features. Table 3.1 shows a simple model with seven features, each defined by an address function and an attribute function. We use the notation STK[i] and BUF[i] for the ith word in the stack and in the buffer, respectively,[7] and we use LDEP(w) and RDEP(w) for the farthest child of w to the left and to the right, respectively. The attribute functions used are FORM for word form and DEPREL for dependency label. In figure 3.5, we show how the value of the feature vector changes as we go through the configurations of the transition sequence in figure 3.2.[8]

Although the feature model defined in figure 3.1 is quite sufficient to build a working parser, a more complex model is usually required to achieve good parsing accuracy. To give an idea of the complexity involved, table 3.2 depicts a model that is more representative of state-of-the-art parsing systems. In table 3.2, rows represent address functions, defined using the same operators as in the earlier example, while columns represent attribute functions, which now also include LEMMA (for

[7] Note that indexing starts at 0, so that STK[0] is the word on top of the stack, while BUF[0] is the first word in the buffer.
[8] The special value NULL is used to indicate that a feature is undefined in a given configuration.

Table 3.2: Typical feature model for transition-based parsing with rows representing address functions, columns representing attribute functions, and cells with + representing features.

	Attributes				
Address	**FORM**	**LEMMA**	**POSTAG**	**FEATS**	**DEPREL**
STK[0]	+	+	+	+	
STK[1]			+		
LDEP(STK[0])					+
RDEP(STK[0])					+
BUF[0]	+	+	+	+	
BUF[1]	+		+		
BUF[2]			+		
BUF[3]			+		
LDEP(BUF[0])					+
RDEP(BUF[0])					+

lemma or base form) and FEATS (for morphosyntactic features in addition to the basic part of speech). Thus, each cell represents a possible feature, obtained by composing the corresponding address function and attribute function, but only cells containing a + sign correspond to features present in the model.

We have focused in this section on configurational word features, i.e., features that can be defined by the composition of an address function and an attribute function, since these are the most important features in transition-based parsing. In principle, however, features can be defined over any properties of a configuration that are believed to be important for predicting the correct transition. One type of feature that has often been used is the *distance* between two words, typically the word on top of the stack and the first word in the input buffer. This can be measured by the number of words intervening, possibly restricted to words of a certain type such as verbs. Another common type of feature is the number of children of a particular word, possibly divided into left children and right children.

3.3.2 TRAINING DATA

Once we have defined our feature representation, we want to learn to predict the correct transition $o(c)$, for any configuration c, given the feature representation $\mathbf{f}(c)$ as input. In machine learning terms, this is a straightforward *classification* problem, where the instances to be classified are (feature representations of) configurations, and the classes are the possible transitions (as defined by the transition system). In a supervised setting, the training data should consist of instances labeled with their correct class, which means that our training instances should have the form $(\mathbf{f}(c), t)$ $(t = o(c))$. However, this is not the form in which training data are directly available to us in a treebank.

In section 2.2, we characterized a training set $\mathcal{D}$ for supervised dependency parsing as consisting of sentences paired with their correct dependency trees:

$$\mathcal{D} = \{(S_d, G_d)\}_{d=0}^{|\mathcal{D}|}$$

In order to train a classifier for transition-based dependency parsing, we must therefore find a way to derive from $\mathcal{D}$ a new training set $\mathcal{D}'$, consisting of configurations paired with their correct transitions:

$$\mathcal{D}' = \{(\mathbf{f}(c_d), t_d)\}_{d=0}^{|\mathcal{D}'|}$$

Here is how we construct $\mathcal{D}'$ given $\mathcal{D}$:

- For every instance $(S_d, G_d) \in \mathcal{D}$, we first construct a transition sequence $C_{0,m}^d = (c_0, c_1, \ldots, c_m)$ such that
  1. $c_0 = c_0(S_d)$,
  2. $G_d = (V_d, A_{c_m})$.
- For every non-terminal configuration $c_i^d \in C_{0,m}^d$, we then add to $\mathcal{D}'$ an instance $(\mathbf{f}(c_i^d), t_i^d)$, where $t_i^d(c_i^d) = c_{i+1}^d$.

This scheme presupposes that, for every sentence S_d with dependency tree G_d, we can construct a transition sequence that results in G_d. Provided that all dependency trees are projective, we can do this using the parsing algorithm defined in section 3.2 and relying on the dependency tree $G_d = (V_d, A_d)$ to compute the oracle function in line 3 as follows:

$$o(c = (\sigma, \beta, A)) = \begin{cases} \textsc{Left-Arc}_r & \text{if } (\beta[0], r, \sigma[0]) \in A_d \\ \textsc{Right-Arc}_r & \text{if } (\sigma[0], r, \beta[0]) \in A_d \text{ and, for all } w, r', \\ & \text{if } (\beta[0], r', w) \in A_d \text{ then } (\beta[0], r', w) \in A \\ \textsc{Shift}_r & \text{otherwise} \end{cases}$$

The first case states that the correct transition is $\textsc{Left-Arc}_r$ if the correct dependency tree has an arc from the first word $\beta[0]$ in the input buffer to the word $\sigma[0]$ on top of the stack with dependency label r. The second case states that the correct transition is $\textsc{Right-Arc}_r$ if the correct dependency tree has an arc from $\sigma[0]$ to $\beta[0]$ with dependency label r – but only if all the outgoing arcs from $\beta[0]$ (according to G_d) have already been added to A. The extra condition is needed because, after the $\textsc{Right-Arc}_r$ transition, the word $\beta[0]$ will no longer be in either the stack or the buffer, which means that it will be impossible to add more arcs involving this word. No corresponding condition is needed for the $\textsc{Left-Arc}_r$ case since this will be satisfied automatically as long as the correct dependency tree is projective. The third and final case takes care of all remaining configurations, where $\textsc{Shift}$ has to be the correct transition, including the special case where the stack is empty.

3.3.3 CLASSIFIERS

Training a classifier on the set $\mathcal{D}' = \{(\mathbf{f}(c_d), t_d)\}_{d=0}^{|\mathcal{D}'|}$ is a standard problem in machine learning, which can be solved using a variety of different learning algorithms. We will not go into the details of how to do this but limit ourselves to some observations about two of the most popular methods in transition-based dependency parsing: memory-based learning and support vector machines.

Memory-based learning and classification is a so-called lazy learning method, where learning basically consists in storing the training instances while classification is based on similarity-based reasoning (Daelemans and Van den Bosch, 2005). More precisely, classification is achieved by retrieving the k most similar instances from memory, given some similarity metric, and extrapolating the class of a new instance from the classes of the retrieved instances. This is usually called k nearest neighbor classification, which in the simplest case amounts to taking the majority class of the k nearest neighbors although there are a number of different similarity metrics and weighting schemes that can be used to improve performance. Memory-based learning is a purely discriminative learning technique in the sense that it maps input instances to output classes without explicitly computing a probability distribution over outputs or inputs (although it is possible to extract metrics that can be used to estimate probabilities). One advantage of this approach is that it can handle categorical features as well as numerical ones, which means that feature vectors for transition-based parsing can be represented directly as shown in section 3.3.1 above, and that it handles multi-class classification without special techniques. Memory-based classifiers are very efficient to train, since learning only consists in storing the training instances for efficient retrieval. On the other hand, this means that most of the computation must take place at classification time, which can make parsing inefficient, especially with large training sets.

Support vector machines are max-margin linear classifiers, which means that they try to separate the classes in the training data with the widest possible margin (Vapnik, 1995). They are especially powerful in combination with kernel functions, which in essence can be used to transform feature representations to higher dimensionality and thereby achieve both an implicit feature combination and non-linear classification. For transition-based parsing, polynomial kernels of degree 2 or higher are widely used, with the effect that pairs of features in the original feature space are implicitly taken into account. Since support vector machines can only handle numerical features, all categorical features need to be transformed into binary features. That is, a categorical feature with m possible values is replaced with m features with possible values 0 and 1. The categorical feature assuming its ith value is then equivalent to the ith binary feature having the value 1 while all other features have the value 0. In addition, support vector machines only perform binary classification, but there are several techniques for solving the multi-class case. Training can be computationally intensive for support vector machines with polynomial kernels, so for large training sets special techniques often must be used to speed up training. One commonly used technique is to divide the training data into smaller bins based on the value of some (categorical) feature, such as the part of speech of the word on top of the stack. Separate classifiers are trained for each bin, and only one of them is invoked for a given configuration during parsing (depending on the value of the feature

Transition		Preconditions
LEFT-ARC$_r$	$(\sigma \mid w_i, w_j \mid \beta, A) \Rightarrow (\sigma, w_j \mid \beta, A \cup \{(w_j, r, w_i)\})$	$(w_k, r', w_i) \notin A$ $i \neq 0$
RIGHT-ARC$_r$	$(\sigma \mid w_i, w_j \mid \beta, A) \Rightarrow (\sigma \mid w_i \mid w_j, \beta, A \cup \{(w_i, r, w_j)\})$	
REDUCE	$(\sigma \mid w_i, \beta, A) \Rightarrow (\sigma, \beta, A)$	$(w_k, r', w_i) \in A$
SHIFT	$(\sigma, w_i \mid \beta, A) \Rightarrow (\sigma \mid w_i, \beta, A)$	

Figure 3.6: Transitions for arc-eager shift-reduce dependency parsing.

used to define the bins). Support vector machines with polynomial kernels currently represent the state of the art in terms of accuracy for transition-based dependency parsing.

3.4 VARIETIES OF TRANSITION-BASED PARSING

So far, we have considered a single transition system, defined in section 3.1, and a single, deterministic parsing algorithm, introduced in section 3.2. However, there are many possible variations on the basic theme of transition-based parsing, obtained by varying the transition system, the parsing algorithm, or both. In addition, there are many possible learning algorithms that can be used to train classifiers, a topic that was touched upon in the previous section. In this section, we will introduce some alternative transition systems (section 3.4.1) and some variations on the basic parsing algorithm (section 3.4.2). Finally, we will discuss how non-projective dependency trees can be processed even if the underlying transition system only derives projective dependency trees (section 3.5).

3.4.1 CHANGING THE TRANSITION SYSTEM

One of the peculiarities of the transition system defined earlier in this chapter is that right dependents cannot be attached to their head until all their dependents have been attached. As a consequence, there may be uncertainty about whether a RIGHT-ARC$_r$ transition is appropriate, even if it is certain that the first word in the input buffer should be a dependent of the word on top of the stack. This problem is eliminated in the *arc-eager* version of this transition system, defined in figure 3.6. In this system, which is called arc-eager because all arcs (whether pointing to the left or to the right) are added as soon as possible, the RIGHT-ARC$_r$ is redefined so that the dependent word w_j is pushed onto the stack (on top of its head w_i), making it possible to add further dependents to this word. In addition, we have to add a new transition REDUCE, which makes it possible to pop the dependent word from the stack at a later point in time, and which has as a precondition that the word on the top of the stack already has a head, i.e., that the arc set contains an arc (w_k, r', w_i) for some k

Transition	Configuration		
	([ROOT],	[Economic, . . . , .],	∅)
SH ⇒	([ROOT, Economic],	[news, . . . , .],	∅)
LA_{ATT} ⇒	([ROOT],	[news, . . . , .],	$A_1 = \{(\text{news, ATT, Economic})\}$)
SH ⇒	([ROOT, news],	[had, . . . , .],	A_1)
LA_{SBJ} ⇒	([ROOT],	[had, . . . , .],	$A_2 = A_1 \cup \{(\text{had, SBJ, news})\}$)
RA_{PRED} ⇒	([ROOT, had],	[little, . . . , .],	$A_3 = A_2! \cup \{(\text{ROOT, PRED, had})\}$)
SH ⇒	([ROOT, had, little],	[effect, . . . , .],	A_3)
LA_{ATT} ⇒	([ROOT, had],	[effect, . . . , .],	$A_4 = A_3 \cup \{(\text{effect, ATT, little})\}$)
RA_{OBJ} ⇒	([ROOT, had, effect],	[on, . . . , .],	$A_5 = A_4 \cup \{(\text{had, OBJ, effect})\}$)
RA_{ATT} ⇒	([ROOT, . . . on],	[financial, markets, .],	$A_6 = A_5 \cup \{(\text{effect, ATT, on})\}$)
SH ⇒	([ROOT, . . . , financial],	[markets, .],	A_6)
LA_{ATT} ⇒	([ROOT, . . . on],	[markets, .],	$A_7 = A_6 \cup \{(\text{markets, ATT, financial})\}$)
RA_{PC} ⇒	([ROOT, . . . , markets],	[.],	$A_8 = A_7 \cup \{(\text{on, PC, markets})\}$)
RE ⇒	([ROOT, . . . , on],	[.],	A_8)
RE ⇒	([ROOT, . . . , effect],	[.],	A_8)
RE ⇒	([ROOT, had],	[.],	A_8)
RA_{PU} ⇒	([ROOT, . . . , .],	[ ],	$A_9 = A_8 \cup \{(\text{had, PU, .})\}$)

Figure 3.7: Arc-eager transition sequence for the English sentence in figure 1.1 (LA_r = LEFT-ARC$_r$, RA_r = RIGHT-ARC$_r$, RE = REDUCE, SH = SHIFT).

and r' (where w_i is the word on top of the stack).[9] To further illustrate the difference between the two systems, figure 3.7 shows the transition sequence needed to parse the sentence in figure 1.1 in the arc-eager system (cf. figure 3.2). Despite the differences, however, both systems are sound and complete with respect to the class of projective dependency trees (or forests that can be turned into trees, to be exact), and both systems have linear time and space complexity when coupled with the deterministic parsing algorithm formulated in section 3.2 (Nivre, 2008). As parsing models, the two systems are therefore equivalent with respect to the Γ and h components but differ with respect to the $\boldsymbol{\lambda}$ component, since the different transition sets give rise to different parameters that need to be learned from data.

Another kind of variation on the basic transition system is to add transitions that will allow a certain class of non-projective dependencies to be processed. Figure 3.8 shows two such transitions called NP-LEFT$_r$ and NP-RIGHT$_r$, which behave exactly like the ordinary LEFT-ARC$_r$ and RIGHT-ARC$_r$ transitions, except that they apply to the second word from the top of the stack and treat the top word as a context node that is unaffected by the transition. Unless this context node is later attached to the head of the new arc, the resulting tree will be non-projective. Although this system cannot

[9] Moreover, we have to add a new precondition to the LEFT-ARC$_r$ transition to prevent that it applies when the word on top of the stack already has a head, a situation that could never arise in the old system. The precondition rules out the existence of an arc (w_k, r', w_i) in the arc set (for any k and r').

Transition		Precondition
NP-LEFT$_r$	$(\sigma \mid w_i \mid w_k, w_j \mid \beta, A) \Rightarrow (\sigma \mid w_k, w_j \mid \beta, A \cup \{(w_j, r, w_i)\})$	$i \neq 0$
NP-RIGHT$_r$	$(\sigma \mid w_i \mid w_k, w_j \mid \beta, A) \Rightarrow (\sigma \mid w_i, w_k \mid \beta, A \cup \{(w_i, r, w_j)\})$	

Figure 3.8: Added transitions for non-projective shift-reduce dependency parsing.

cope with arbitrary non-projective dependency trees, it can process many of the non-projective constructions that occur in natural languages (Attardi, 2006).

In order to construct a transition system that can handle arbitrary non-projective dependency trees, we can modify not only the set of transitions but also the set of configurations. For example, if we define configurations with two stacks instead of one, we can give a transition-based account of the algorithms for dependency parsing discussed by Covington (2001). With an appropriate choice of transitions, we can then define a system that is sound and complete with respect to the class $\mathcal{G}_S$ of arbitrary dependency for a given sentence S. The space complexity for deterministic parsing with an oracle remains $O(n)$ but the time complexity is now $O(n^2)$. To describe this system here would take us too far afield, so the interested reader is referred to Nivre (2008).

3.4.2 CHANGING THE PARSING ALGORITHM

The parsing algorithm described in section 3.2 performs a greedy, deterministic search for the optimal transition sequence, exploring only a single transition sequence and terminating as soon as it reaches a terminal configuration. Given one of the transition systems described so far, this happens after a single left-to-right pass over the words of the input sentence. One alternative to this single-pass strategy is to perform multiple passes over the input while still exploring only a single path through the transition system in each pass. For example, given the transition system defined in section 3.1, we can reinitialize the parser by refilling the buffer with the words that are on the stack in the terminal configuration and keep iterating until there is only a single word on the stack or no new arcs were added during the last iteration. This is essentially the algorithm proposed by Yamada and Matsumoto (2003) and commonly referred to as Yamada's algorithm. In the worst case, this may lead to $n - 1$ passes over the input, each pass taking $O(n)$ time, which means that the total running time is $O(n^2)$, although the worst case almost never occurs in practice.

Another variation on the basic parsing algorithm is to relax the assumption of determinism and to explore more than one transition sequence in a single pass. The most straightforward way of doing this is to use *beam search*, that is, to retain the k most promising partial transition sequences after each transition step. This requires that we have a way of scoring and ranking all the possible transitions out of a given configuration, which means that learning can no longer be reduced to a pure classification problem. Moreover, we need a way of combining the scores for individual transitions

in such a way that we can compare transition sequences that may or may not be of the same length, which is a non-trivial problem for transition-based dependency parsing. However, as long as the size of the beam is bounded by a constant k, the worst-case running time is still $O(n)$.

3.5 PSEUDO-PROJECTIVE PARSING

Most of the transition systems that are used for classifier-based dependency parsing are restricted to projective dependency trees. This is a serious limitation given that linguistically adequate syntactic representations sometimes require non-projective dependency trees. In this section, we will therefore introduce a complementary technique that allows us to derive non-projective dependency trees even if the underlying transition system is restricted to dependency trees that are strictly projective. This technique, known as *pseudo-projective parsing*, consists of four essential steps:

1. Projectivize dependency trees in the training set while encoding information about necessary transformations in augmented arc labels.
2. Train a projective parser on the transformed training set.
3. Parse new sentences using the projective parser.
4. Deprojectivize the output of the projective parser, using heuristic transformations guided by augmented arc labels.

The first step relies on the fact that it is always possible to transform a non-projective dependency tree into a projective tree by substituting each non-projective arc (w_i, r, w_j) by an arc $(\text{ANC}(w_i), r', w_j)$, where $\text{ANC}(w_i)$ is an ancestor of w_i such that the new arc is projective. In a dependency tree, such an ancestor must always exist since the ROOT node will always satisfy this condition even if no other node does.[10] However, to make a minimal transformation of the non-projective tree, we generally prefer to let $\text{ANC}(w_i)$ be the *nearest* ancestor (from the original head w_i) such that the new arc is projective.

We will illustrate the projectivization transformation with respect to the non-projective dependency tree in figure 2.1, repeated in the top half of figure 3.9. This tree contains two non-projective arcs: *hearing* $\overset{\text{ATT}}{\rightarrow}$ *on* and *scheduled* $\overset{\text{TMP}}{\rightarrow}$ *today*. Hence, it can be projectivized by replacing these arcs with arcs that attach both *on* and *today* to *is*, which in both cases is the head of the original head. However, to indicate that these arcs do not belong to the true, non-projective dependency tree, we modify the arc labels by concatenating them with the label going into the original head: SBJ:ATT and VC:TMP. Generally speaking, a label of the form HEAD:DEP signifies that the dependent has the function DEP and was originally attached to a head with function HEAD. Projectivizing the tree with this type of encoding gives the tree depicted in the bottom half of figure 3.9.

Given that we have projectivized all the dependency trees in the training set, we can train a projective parser as usual. When this parser is used to parse new sentences, it will produce dependency

[10] I.e., for any arc (w_0, r, w_j) in a dependency tree, it must be true that $w_0 \rightarrow^* w_k$ for all $0 < k < j$.

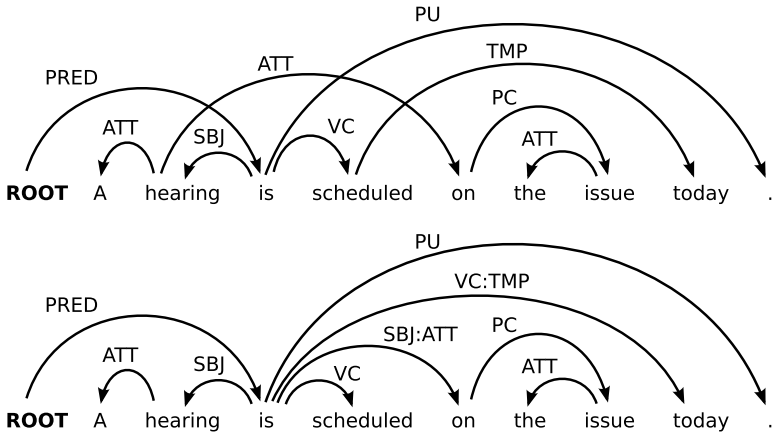

Figure 3.9: Projectivization of a non-projective dependency tree.

trees that are strictly projective as far as the tree structure is concerned, but where arcs that need to be replaced in order to recover the correct non-projective tree are labeled with the special, augmented arc labels. These trees, which are said to be pseudo-projective, can then be transformed into the desired output trees by replacing every arc of the form $(w_i, \text{HEAD:DEP}, w_j)$ by an arc $(\text{DESC}(w_i), \text{DEP}, w_j)$, where $\text{DESC}(w_i)$ is a descendant of w_i with an ingoing arc labeled HEAD. The search for $\text{DESC}(w_i)$ can be made more or less sophisticated, but a simple left-to-right, breadth-first search starting from w_i is usually sufficient to correctly recover more than 90% of all non-projective dependencies found in natural language (Nivre and Nilsson, 2005).

The main advantage of the pseudo-projective technique is that it in principle allows us to parse sentences with arbitrary non-projective dependency trees in linear time, provided that projectivization and deprojectivization can also be performed in linear time. Moreover, as long as the base parser is guaranteed to output a dependency tree (or a dependency forest that can be automatically transformed into a tree), the combined system is sound with respect to the class $\mathcal{G}_S$ of non-projective dependency trees for a given sentence S. However, one drawback of this technique is that it leads to an increase in the number of distinct dependency labels, which may have a negative impact on efficiency both in training and in parsing (Nivre, 2008).

3.6 SUMMARY AND FURTHER READING

In this chapter, we have shown how parsing can be performed as greedy search through a transition system, guided by treebank-induced classifiers. The basic idea underlying this approach can be traced back to the 1980s but was first applied to data-driven dependency parsing by Kudo and Matsumoto (2002), who proposed a system for parsing Japanese, where all dependencies are head-final. The approach was generalized to allow mixed headedness by Yamada and Matsumoto (2003), who applied

it to English with state-of-the-art results. The latter system essentially uses the transition system defined in section 3.1, together with an iterative parsing algorithm as described in section 3.4.2, and classifiers trained using support vector machines.

The arc-eager version of the transition system, described in section 3.4.1, was developed independently by Nivre (2003) and used to parse Swedish (Nivre et al., 2004) and English (Nivre and Scholz, 2004) in linear time using the deterministic, single-pass algorithm formulated in section 3.2. An in-depth description of this system, sometimes referred to as Nivre's algorithm, can be found in Nivre (2006b) and a large-scale evaluation, using data from ten different languages, in Nivre et al. (2007). Early versions of this system used memory-based learning but more accurate parsing has later been achieved using support vector machines (Nivre et al., 2006).

A transition system that can handle restricted forms of non-projectivity while preserving the linear time complexity of deterministic parsing was first proposed by Attardi (2006), who extended the system of Yamada and Matsumoto and combined it with several different machine learning algorithms including memory-based learning and logistic regression. The pseudo-projective parsing technique was first described by Nivre and Nilsson (2005) but is inspired by earlier work in grammar-based parsing by Kahane et al. (1998). Systems that can handle arbitrary non-projective trees, inspired by the algorithms originally described by Covington (2001), have recently been explored by Nivre (2006a, 2007).

Transition-based parsing using different forms of beam search, rather than purely deterministic parsing, has been investigated by Johansson and Nugues (2006, 2007b), Titov and Henderson (2007a,b), and Duan et al. (2007), among others, while Cheng et al. (2005) and Hall et al. (2006) have compared the performance of different machine learning algorithms for transition-based parsing. A general framework for the analysis of transition-based dependency-based parsing, with proofs of soundness, completeness and complexity for several of the systems treated in this chapter (as well as experimental results) can be found in Nivre (2008).